

基于本项目的复刻学习教程

以 aubo_e5_jazzy_ws（桃果采摘机器人：固定座 AUBO E5 + RGB-D + 三把剪切手工具族）的真实源码为教材，从零到整栈的渐进式学习路径。

v1 · 2026 年 9 月 29 日（配套《复刻知识清单与书单.md》v2；书单与获取渠道见《书单与正版获取渠道.md》）

0. 这份教程怎么用

0.1 学习哲学

- **代码即教材**：每个阶段都「先读本仓代码 → 再查官方文档印证 → 再动手」。孤立看书容易忘，绑定真实文件才会。
- **纯核先行**：本仓把业务逻辑写成零 ROS 纯核（不 import rclpy/rclcpp），节点只做接线。学习时优先读纯核——不需要起 ROS 就能读懂、能用 pytest 验证你的理解。
- **mock 自由、真机授权**：所有练习都在 hardware_mode:=mock 下进行，不起真机。真机相关只学概念与边界（S6/第 9 章）。
- **测完即停**：任何练习结束，Ctrl+C 停栈后必须 pgrep -af 'ros2 launch|component_container|extrinsics_publisher|ros2 run' 复核无残留（MUST 纪律，从第一天养成）。

0.2 前置环境

```
source /opt/ros/jazzy/setup.bash
cd ~/Desktop/aubo_e5_jazzy_ws
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release -
DCMAKE_EXPORT_COMPILE_COMMANDS=ON
source install/setup.bash
```

Python 第三方在 venv（aubo_py3.12，numpy==1.26.4 硬约束）；ROS 依赖走 apt。

0.3 全栈鸟瞰（先记住这张图，后面每阶段填一块）

harvest_system.launch.py（peach_bringup，预检拒旧实例）

aubo_e5_bringup	驱动只读；mock=GenericSystem+JTC
serial_imu	不进 lifecycle
imu_follow_servo	仅 tool_profile=adaptive_shear_v1
peach_harvester.brain	一进程三节点：
peach_scene_perception_node	（L1 感知：一帧→观测）
peach_target_reconstruction_node	（L1 重建：多帧→模型
+GraspDecision)	

peach_supervisor	(调度：FSM/选果/账本)
peach_arm	C++ 技能：ExecuteTarget 阶段序列
peach_observability	8090 观测 + 会话 bag (只读)
peach_lifecycle_manager	nav2_lm 托管：场景→重建→技能→调度

数据单向（背下来，这是全仓最重要的不变量）：

```

RGB-D → 感知(事实) → 观测/初值
      ↓
      重建(模型 + GraspDecision)
      ↓
      调度(选果/FSM/账本) --ExecuteTarget/Survey--> 臂(运动)
      ↑ CheckReachability 是唯一反向 IK 查询
  
```

禁止：感知发运动；技能写账本；调度把 initial_pose/grasp_decision 当选果源。

0.4 导航地图（学会查，比记住更重要）

想知道什么	打开
怎么改代码（MUST/DEFAULT/KEEP/UNWIND）	AGENTS.md（仓库根）
现在跑什么（SNAPSHOT）	docs/ architecture.md、docs/ io.md、docs/testing.md
图名/QoS/TF 权威	src/ peach_interfaces/ config/ interface_manifest.yaml
某文件干什么	.agents/skills/ peach/modules.md（AI 源码地图，人读同样好用）
一批怎么跑	.agents/skills/ peach/harvest-flow.md
一帧怎么变观测	.agents/skills/ peach/vision.md
一颗怎么执行	.agents/skills/ peach/arm.md
历史过程	docs/testing- log.md、docs/ REFACTORING.md（只追加）

1. 阶段总览

阶段	主题	参考时长	过关产出
S0	工具底座：把仓库跑起来	1 周	构建+单包测试全绿
S1	ROS 2 通信骨架	1–2 周	读懂 IDL+manifest, 会用五件套探针
S2	感知入门：一帧的旅程	2–3 周	说清一帧→观测的八步
S3	感知数学：像素到 GraspDecision	4 周	复算 χ^2 门与工具预算
S4	调度与状态机	1–2 周	跟踪一次 RunHarvest 全命令链
S5	C++ 与运动技能	4–6 周	讲清授权门与阶段序列
S6	整栈与硬件边界	1 周	画出安全三层, mock 冒烟全绿
S7	测试与验证	1–2 周	跑一轮 launch_testing 三档域
S8	进阶选修：工具族/仿真/战役	2 周+	自加一个 tool_profile 过冒烟

时长按业余时间估算；顺序即依赖序，跳过即卡壳。

S0 工具底座：把仓库跑起来（≈1 周）

目标：会构建、会跑测试、会读提交历史、懂目录语义。

教材：《Pro Git》第 1–3 章；colcon 官方文档 Getting Started；docs/testing.md 全文。

代码导读：

- 仓库根 AGENTS.md 的「0. 怎么用」与「9. 运行」两章（第一周唯一必读的仓内文档）

- `ls build/ install/ log/` 认识三个生成目录（不入库）；`.clangd` 与 `build/compile_commands.json` 的关系

动手练习：

1. 完整构建一遍（命令见 0.2），记录耗时。
2. `colcon test --packages-select peach_common && colcon test-result --verbose`——第一次绿灯。
3. `git log --oneline -30`：观察「小修攒批、里程碑提交」的纪律与提交说明写法（每条都带为什么）。
4. 刻意练习一次清进程：起一个 `ros2 topic list` 探针后 `pgrep -af ros2`，确认退出无残留。

过关判据：能口头说出 `build/install/log` 各放什么、`--packages-select` 怎么用、为什么要 `CMAKE_EXPORT_COMPILE_COMMANDS=ON`。

S1 ROS 2 通信骨架：26 个包怎么说话（≈1-2 周）

目标：掌握四种通信通道、QoS、Lifecycle、launch，读懂本仓契约包。

教材：[docs.ros.org Jazzy 的 Topics/Services/Actions/Parameters/Understanding-QoS/Managed-Nodes/Launch](https://docs.ros.org/Jazzy/Topics/Services/Actions/Parameters/Understanding-QoS/Managed-Nodes/Launch) 各章；《动手学 ROS2》对应篇（Humble 但概念通用）。

代码导读（`src/peach_interfaces/`）：

1. `msg/HarvestState.msg`——带 Header、RELIABLE+transient_local 闷锁的批次状态
2. `msg/GraspDecision.msg`——分档授权字段（`sleeve_capability/radial_margin_m/cut_capability/axial_margin_m/pregrasp_verified`），注意字段行内注释写单位与 frame
3. `action/ExecuteTarget.action`——goal/result/feedback 三段，outcome 常量（SUCCEEDED=0...CANCELED=4）
4. `config/interface_manifest.yaml`——谁发谁订、QoS，权威高于 IDL 文件头注释（头注释常残留旧图名 `peach_executor`）
5. `scripts/check_interface_manifest.py`——manifest 与实现双向核对，漂移即失败

动手练习：

- 1. 起 mock 栈：ros2 launch peach_bringup harvest_system.launch.py hardware_mode:=mock camera_enabled:=false
- 2. 五件套探针：ros2 node list/ros2 topic list/ros2 action list/ros2 lifecycle get /peach_supervisor/ros2 topic echo /joint_states --once（六关节名与 MUST 关节序一致）
- 3. ros2 topic info -v /peach_supervisor/state——亲眼看 QoS 与发布订阅两端
- 4. ros2 interface show peach_interfaces/msg/GraspDecision 与 manifest 对读
- 5. 练习完停栈 + pgrep 复核（每次都做，下不赘述）

过关判据：能回答——调度为什么只订 target_observations 与 managed_nodes_activated 两条跨包流？initial_pose 为什么只能进重建？RELIABLE 订户为什么收不到 BEST Effort 发布？

S2 感知入门：一帧图像的旅程（≈2-3 周）

目标：读懂感知节点的八步管线；掌握图像同步、深度单位、质量门。

教材：动手学 ROS2 通信与传感器篇；OpenCV 官方 Read/Display/Save；ultralytics 官方 Quickstart；message_filters 官方页。

代码导读（src/peach_harvester/peach_harvester/vision/scene_perception/，按此顺序）：

序	文件	看什么
1	scene_perception_node.py	Lifecycle 壳：订阅→decode→查 TF→发布，业务全在纯核
2	pipeline.py	process(SyncedRgbd) → PerceptionResult 门面：八步的目录页
3	inference.py	YOLO 检测去重 + MobileSAM 实例掩膜；无 SAM 时深度带连通域降级（须标 mask_source）
4	pose_pipelines.py	RobustBagPosePipeline（袋=圆柱 RANSAC 定轴）/RobustFruitPosePipeline（果=球拟合+梗洼定向）
5	identity.py	TargetRegistry：世界系 $\chi^2 \leq 9$ 门

序	文件	看什么
		+匈牙利 1-1+类别+歧义比 1.2 ; clear() 不复位计数
6	image_gates.py	静止门/有效深度占比/贴边框不攒确认
7	msg_builders.py	IDL 组装：观测怎么变成消息

八步口诀：同步 (slop 0.05s) → 深度毫米裁前景 → 检测 → 分割 → 拟合位姿 → 单帧状态 (ACCEPT/REOBSERVE/REJECT) → 身份分配 → 锁定发布。

关键认知：单帧状态只当初值/可视化，不得据此接触；TF 精确 stamp 失败则 tf_status!='ok'，不进身份链。

动手练习：

1. colcon test --packages-select peach_harvester, 打开 test/ 里感知纯核的表驱动用例，改一个断言看它怎么红。
2. 手算一次单位换算：Percipio 深度 uint16 值 850 → 0.85 m；写三行 numpy 完成掩膜反投影。
3. 读 docs/io.md 感知发布通道表，与 msg_builders.py 对读。

过关判据：能对着 pipeline.py 白板讲出八步，并说出「为什么贴边框不攒确认」。

S3 感知数学：从像素到 GraspDecision (≈4 周，最难)

目标：掌握 3D 变换/拟合/ICP/TSDF/预算决策；这是全项目数学最重的一层。

教材：《视觉 SLAM 十四讲》第 2-6 讲（变换/李群李代数可略读）+ 第 7 讲（相机模型）+ 点云与 ICP 章；《现代机器人学》刚体运动章；SciPy spatial.transform 文档；Open3D 官方 ICP/TSDF 示例。

代码导读 (src/peach_harvester/peach_harvester/vision/)：

1. common/geometry.py——纯函数几何层，先读懂这里的旋转/变换约定（体轴 x 前 y 左 z 上；光学系 z 前）
2. target_reconstruction/reconstruction_core.py——帧环/自动机/发布宿主：ReconstructionSession 会话态
3. target_reconstruction/capture.py——采帧门 (IDLE/COLLECTING)，停走式相机节拍

4. `target_reconstruction/integrate.py`——锁外查 **base←camera** 精确时刻 TF (失败跳帧, 禁止运动中 latest) → 质量门五道 `StrictMaskGate` → FK 到 base → 有界 ICP (只修小刚性误差) → 在线 TSDF
5. `target_reconstruction/refine.py` + `refit_orchestrator.py`——
finalize: 抽网格+柱/球 refit 融合
6. `common/tool_budget.py`——动态径向/轴向预算: 套袋看
`sleeve_capability/radial_margin_m>0`; 剪切看
`cut_capability/axial_margin_m>0`+预抓取残差 → 汇成 `GraspDecision`
7. 手眼链路: `src/aubo_hand_eye_calibration/` (Web 引导采集, `hand_eye/active.yaml` 是外参事实源) → 唯一发布者
`extrinsics_publisher` 发 `wrist3→camera_link`

动手练习:

1. 十四讲前 6 讲课后题选做一半; 用 `scipy.spatial.transform.Rotation` 复算一次「光学系→体轴」旋转 (z 前 x 右 y 下 → x 前 y 左 z 上)。
2. 手算身份门: 两观测差向量的马氏距离平方=10.2, χ^2 门=9, 判匹配还是新建? (答: 新建, 超 3σ)
3. 读 `tool_budget.py` 的预算公式, 代入
`aubo_description/config/adaptive_shear_v1.yaml` 的
`D_inner/L_insert/L_blade` 算一次径向/轴向余量的正负含义。
4. 用 Open3D 官方示例跑一遍 ICP, 再回来看 `integrate.py` 的「有界」体现在哪 (答案: 限制单次修正幅度, 只修小刚性误差, 不吞大错)。

过关判据: 能回答——重建为什么禁止 latest TF? `valid_until` 为什么冻结不续签?
`GraspDecision.allowed` 与臂侧分档校验是什么关系 (提示: `allowed` 只是汇总展示, 强制点在臂侧 `authorizeStage`) ?

S4 调度与状态机: 一批 RunHarvest 怎么跑 (≈1-2 周)

目标: 掌握表驱动 FSM、reducer、选果策略、批次策略与账本。

教材: 《代码整洁之道》函数章 (为什么表驱动优于 if 链); 本仓
`.agents/skills/peach/harvest-flow.md` (当讲义用); launch 官方 action 章复习。

代码导读 (`src/peach_harvester/peach_harvester/supervisor/`):

1. harvest_fsm.py——_TABLE 事件→命令纯核表：先背「WAITING_READY+RUN_REQUESTED→DISCOVERY/NAVIGATE」这条主干，再读全表
2. domain/reducer.py——三维状态+世代/事务去重（重复事件不重放）
3. executor_node.py——命令循环：reaction.command 分发到 _cmd_* handler 表（NAVIGATE/SURVEY/BEGIN_SCENE/WAIT_LOCK/SELECT/DISPATCH/EXECUTE_FULL/NONE）
4. batch.py——next_target 选果：资格谓词（锁定/非裸果/不贴边）→深度∩可达窗→priority 升序+检测框面积降序；超窗进 targets_filtered；跳过是策略不是失败（进 ReworkList）
5. cycle_core/batch_policy.py——采收率门/单果时限/view_policy 两档（VIEW_FAST supervisor 直驱补视 vs VIEW_CONSERVATIVE 发臂 OBSERVE_ONLY）
6. 账本：runs/<request_id>/ledger.json + rework_list.json（json 原子写）

关键认知：批次态只经 harvest_fsm.react/reducer/_apply，节点禁止 self._batch_state=...；接触/PREGRASP 真动过后 recovery_required，须 ACK 才继续。

动手练习：

1. colcon test --packages-select peach_harvester 里的 FSM 用例：加一条「DISCOVERY+NO_TARGET→再 SURVEY 不再 Begin」的表驱动断言。
2. mock 起栈（S1 命令），浏览器开 http://127.0.0.1:8090 观测页，人发一次 RunHarvest（或 ros2 action send_goal），对照 FSM 表在观测页跟一遍状态迁移；ros2 topic echo /peach_supervisor/state 看门锁状态。
3. 练 PAUSED：ControlTask 暂停后观察 EventHold 只暂存第一条非 NONE 命令、RESUME 只释放一次。

过关判据：能不看文档写出一条完整事件链：RUN_REQUESTED→...→SETTLE，并说出 SELECT 前置的三道资格+一道窗。

S5 C++ 与运动技能：ExecuteTarget 的阶段序列（≈4-6 周）

目标：读懂 C++ 技能包的受理门、授权矩阵、接近路径与护栏；这是 C++ 学习的主战场。

教材：《C++ Primer》智能指针/移动语义/并发三选读；《Effective Modern C++》对应条目（本文件夹《现代 C++ 教程》免费垫底）；MoveIt 2 官方 Getting Started+MTC+Pilz 三章；control.ros.org 概念章（只读学）。

代码导读（src/peach_arm/src/, 按此顺序）：

1. manipulation_skills_node.cpp——Lifecycle 壳：订阅/发布/状态 JSON；MoveIt 伴随节点同 executor
2. cycle.cpp——动作受理门（非 Active/周期占用/contact_recovery_required→REJECT；模型七元组非空 presence-check）+ **authorizeStage** 授权矩阵：
 - TRANSIT/PREGRASP → authorizeTransit（使能×Active×robotReady×-cancel）
 - CONTACT/TOOL → motionOutputAllowed + safetyReady + 令牌优先（allowed+target 绑定+valid_until+model_stamp 新鲜度）→ CONTACT 查 radial_margin_m>0；TOOL 再加 axial_margin_m>0+pregrasp_verified+tool_enabled → 无令牌回退快照
3. stages.cpp——executeCycle 阶段序列（当目录页背）：Prepare→(观察)→Finalize→(PREVIEW/READY 分岔)→Reconfirm→MovePregrasp→VerifyPregrasp→(PREGRASP_ONLY 停驻)→Sleeve→Cutter→VerifyCut→ReverseRetreat→Stow→Release→Complete
4. grasp_task.cpp——MTC 装配：接近 v4 主档（Pilz PTP 至冠外正下方→世界垂直 LIN 入冠→沿轴 LIN 进预抓取+20° OrientationConstraint）；套入=沿轴 LIN (shear/bite) 或交 imu_follow (adaptive)
5. motion.cpp / move_to.cpp——MoveIt 接口、拍照位、enables 心跳
6. target_cache.cpp——selected/locked/refined/decision 三索引缓存
7. 纯核门：safety_gate.cpp、quality_gate.cpp、include/peach_arm/trajectory_guard.hpp（绕行累计 12 rad/单轴 6.1 rad 护栏）、grasp_geometry.hpp（果实胶囊仅接近段，套入/撤退豁免）
8. config/peach_arm.yaml——GPL 参数单源（对照 PickNik 文档）

关键认知：execution_enabled 时一次性 execution_armed，周期结束自动解除；8090 调试运动绕不过 authorizeStage；ContactMonitor 腕轴电流默认关，不是急停。

动手练习：

1. `colcon test --packages-select peach_arm` (gtest 已解禁, 纯核门有单测)。
2. `mock` 起栈后在 RViz (MoveIt 已随栈起) 看一次 PREVIEW 规划的可视化; 对照 docs/testing.md 冒烟最小集。
3. 代码走查练习: 在 `cycle.cpp` 找到令牌校验与快照回退两路, 画一张 5 层判定流程图。
4. 读 `acm_policy.hpp`: 哪些物体在哪个阶段豁免碰撞 (答案线索: 接近段查果实胶囊、套入段故意进囊豁免)。

过关判据: 能口头回答——PREGRASP_ONLY 为什么是默认? 令牌与快照双路各自防什么? CONTACT 与 TOOL 档差的三个字段是什么?

S6 整栈与硬件边界: mock 到 real 之间隔着什么 (≈1 周)

目标: 理解整栈装配、mock/real 切换机制、安全三层模型; 只学概念, 不碰真机。

教材: AGENTS.md 第 1-2 章 (MUST 红线+安全分层); ros2_control 官方 getting started 与 mock_components 章; UR ROS2 Driver 文档 (对照为什么本仓 bringup 不起 dashboard)。

代码导读:

1. `src/peach_bringup/launch/harvest_system.launch.py`——预检 (拒叠 RSP/extrinsics)、Include 顺序 (应用 Include 驱动 bringup, 不复制)、`tool_profile` 透传注入
2. `src/peach_bringup/`
`preflight.py`、`lifecycle_flag_bridge.py`、`autostart_client.py`
3. mock 切换: URDF xacro 里 `mock_components/GenericSystem` vs 真机 `SystemInterface` (驱动九包只读, 读 `aubo_e5_bringup` 只为理解, 不改)
4. `src/peach_harvester/peach_harvester/vision/tool_profiles.py` + `src/aubo_description/config/*.yaml`——三把剪切手档案单源与注入链

安全三层 (必须能复述):

层	做什么	不做什么
功能安全 (柜/示教器)	急停 ISO 13850、保护停止、使能装置	不跑在 DDS/用户态
应用护栏 (ROS 2)	使能门 ExecutionAuthority、令牌	不宣称 SIL/功能安全认证

层	做什么	不做什么
	校验、命令超时、bond、规划碰撞	
网络安全 (SROS2)	加密认证权限	不是急停，挡不住臂在动

动手练习：

1. 完整 mock 冒烟一遍 (docs/testing.md 冒烟最小集五条：控制器 active/关节序/TF world→tcp/lifecycle 四节点 Active/再停栈 pgrep)。
2. 跑三把刀逐档冒烟：colcon test --packages-select peach_system_tests (内部三注册 PEACH_LT_TOOL_PROFILE, 隔离域 91/92/93)。
3. 读 .github/workflows/jazzy.yaml：CI 怎么编测、哪些包被忽略。

过关判据：能解释——为什么软件 RobotMoveStop 不得称 e-stop？为什么保护停止解除后禁止 resume 原轨迹？mock 证明了什么、没证明什么？

S7 测试与验证：怎么证明没改坏 (≈1-2 周)

目标：掌握本仓四层测试塔与验证方法学。

教材：docs.ros.org Testing Main+launch_testing；pytest 官方入门；本仓 .agents/skills/peach/modules.md 的 peach_system_tests 节。

代码导读：

1. 各包 test/——纯核表驱动 pytest (test_harvest_fsm.py 是范式)；C++ gtest (peach_arm/test/)
2. src/peach_system_tests/——test_tool_profile_smoke.py (三档域)、replay_oracle.py (接近回放塔, 纯几何, 不消费系统 ID)、perception_sim.py、test_el_supervisor_chain.py
3. scripts/r0_gate.sh + .github/workflows/jazzy.yaml——CI 门
4. 会话产物：runs/<request_id>/ (ledger.json/jsonl/bag MCAP) 与 campaign/*/ (战役=方案 README+脚本+报告)

动手练习：

1. colcon test --packages-select peach_system_tests 全绿 (注意：launch_testing 必须走 colcon 而非裸 ctest, 否则缺 overlay 环境会假红)。
2. 对一个纯核函数写一个「故意写错实现→看测试红→改回绿」的完整循环。

3. 打开任一 `campaign/*/README.md`, 学习「方案→脚本→报告」的战役组织法。

关键认知：`colcon test` 绿 ≠ 采摘方向验收（真机才是最终权威）；`launch_testing` 必须 `isolated ROS_DOMAIN_ID`（不打扰开发机在跑的栈）。

过关判据：能为本仓任一纯核模块设计一个表驱动用例，并说出它落在测试金字塔哪一层。

S8 进阶选修：工具族、仿真矩阵、战役方法（≈2 周+）

三个专题按兴趣选做，也是从「会读」到「会改」的跳板：

1. **工具族全链：**`aubo_description/config/<profile_id>.yaml`（单一事实源：TCP/开口方向/包络/D_inner/L_insert/L_blade）→ `tool_profiles.py` → `launch` 注入 `scene/recon` → `TF wrist3_Link`→`tcp` 对档案 ±2 mm。练习：照抄一份 `yaml` 改名 `my_shear_v1`, 把三处（档案/注册表/冒烟注册）接起来让 `test_tool_profile_smoke` 过。
 2. **合成数据验证：**`src/peach_sim/reconstruction/`（Blender 无头渲染果园场景；852 帧感知验证矩阵 M0-M3、GATE PASS 基线冻结）。练习：读懂 `build_scene.py` 与 GT 清单格式，理解「GT 不消费系统 ID」为什么是验证的金标准。
 3. **战役方法：**`campaign/20260922_dual_tool/` 与 `campaign/20260928_bite_shear/`——怎么把一轮验证组织成可复现的目录（方案冻结→脚本执行→报告落盘→`testing-log` 记档）。
-

9. 真机复刻的硬边界（不可越，先背再学）

1. **未授权不得真机运动或 SetIO。** 授权=人的口头/书面确认=操作员在 `real` 上发起 `launch/` 操作台指令；不是 `yaml` 里的某个 `true`, 不是 8090 按钮。
2. **硬件急停不经 ROS。** 示教器红钮/柜安全回路才是 ISO 13850 急停；一切软件停（`RobotMoveStop/`取消 `action/8090`）都只是应用层停轨。
3. **驱动九包只读**（`hardware/controllers/dashboard/ros2_control.xacro/bringup/controllers.yaml`）；`bringup` 不起 `dashboard`、不远程上电。
4. **测完即停：**任何测试/探针/`bag/`演示结束，停栈并 `pgrep` 复核，挂死按 PID 清。
5. **急停/保护停止后：**先处理现场→示教器复位→取消丢弃在途 `goal`→重新授权下发；**禁止 resume 原轨迹。**
6. **mock 通过只证明接线与使能门逻辑；**碰撞、电流、急停回路、刀、人在环只在真机且需授权。

7. 关节序冻结：shoulder_joint, upperArm_joint, foreArm_joint, wrist1_joint, wrist2_joint, wrist3_joint, 全链一致。
-

10. 学习资源速查

官方文档（主教材）：ROS 2 Jazzy docs.ros.org/en/jazzy · MoveIt 2 moveit.picknik.ai · ros2_control control.ros.org · Open3D open3d.org/docs · NumPy/SciPy · ultralytics GitHub · MDN。

本文件夹：免费 PDF 四本（Pro Git/现代机器人学预印本/现代 C++ 教程/动手学 ROS2）+ 官方配套源码六仓（见《书单与正版获取渠道.md》）。

仓内文档地图：AGENTS.md（怎么改）→ docs/architecture.md（现在是什么，含 C4 图与决策表）→ docs/io.md（图名/QoS/TF）→ docs/testing.md（怎么跑/验收门）→ docs/testing-log.md+docs/REFACTORING.md（只追加过程档）→ .agents/skills/peach/（源码地图六件）→ campaign/（战役实录）。

祝复刻顺利。记住本项目的三条工程信条：数据单向、纯核先行、测完即停。